Solution: Find all Missing Numbers

Problem Statement

We are given an unsorted array containing numbers taken from the range 1 to ‘n’. The array can have duplicates, which means some numbers will be missing. Find all those missing numbers.

Example 1:

```
Input: [2, 3, 1, 8, 2, 3, 5, 1]
Output: 4, 6, 7
Explanation: The array should have all numbers from 1 to 8, due to duplicates 4, 6, and 7 are missing.
```

Example 2:

```
Input: [2, 4, 1, 2]
Output: 3
```

Example 3:

```
Input: [2, 3, 2, 1]
Output: 4
```

Solution

This problem follows the **Cyclic Sort pattern** and shares similarities with Find the `Missing Number` with one difference. In this problem, there can be many duplicates whereas in ‘Find the Missing Number’ there were no duplicates and the range was greater than the length of the array.

However, we will follow a similar approach though as discussed in `Find the Missing Number` to place the numbers on their correct indices. Once we are done with the cyclic sort we will iterate the array to find all indices that are missing the correct numbers.

Code

Here is what our algorithm will look like:

Java

```java
import java.util.*;

class Solution {

  public List<Integer> findNumbers(int[] nums) {
    int i = 0;  // Initialize a pointer for iterating through the array.
    while (i < nums.length) {
      if (nums[i] != nums[nums[i] - 1])  // Check if the current element is not in its correct position.
        swap(nums, i, nums[i] - 1);  // Swap the elements to their correct positions.
      else
        i++;  // If the current element is in the correct position, move to the next element.
    }

    List<Integer> missingNumbers = new ArrayList<>();
    for (i = 0; i < nums.length; i++)
      if (nums[i] != i + 1)  // Check if the element at index 'i' is not in the correct position.
        missingNumbers.add(i + 1);  // Add the missing number to the list.

    return missingNumbers;
  }

  private static void swap(int[] arr, int i, int j) {
    int temp = arr[i];  // Swap two elements in the array.
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The time complexity of the above algorithm is O(n).

Space Complexity

Ignoring the space required for the output array, the algorithm runs in constant space O(1).

← Previous

Find All Missing Numbers (Easy)

Next →

Find The Duplicate Number (Easy)

Mark as Completed